Visualization Tools for Structural Analysis

Final Report

Table of Contents

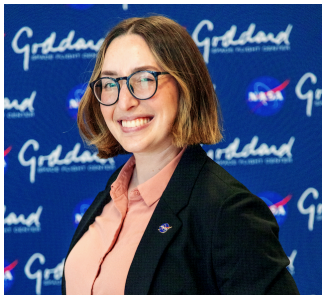
1. Project Information

1.1 Team Information

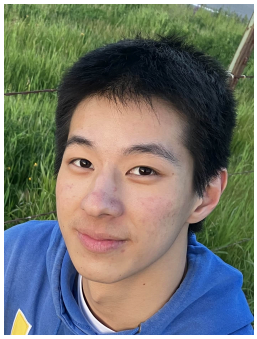
Team Name: SIGMA (Structural Insights and Graphical Model Analysis)
Team # on Canvas: KPFF - Structural Analysis

1.2 Team Members

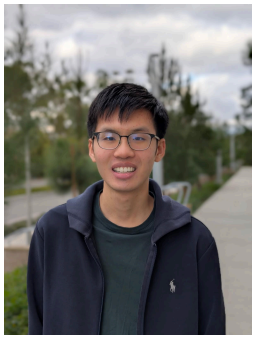
Team Member 1 (Team Lead):
Newton, Morgan
40885576
monewton@uci.edu



Team Member 2:
Young, Peter
55292320
pyoung3@uci.edu



Team Member 3:
Tran, Dillon
80272704
dillonct@uci.edu



Team Member 4:
Dey, Rosnita
77717795
rsdey@uci.edu

Team Member 5:
Liu, James
16169703
jyunrl@uci.edu

Team Member 6:
Rothbard, John
31527828
jrothbar@uci.edu

2. Introduction

2.1 Motivation and Background

KPFF, established in 1960, is a renowned structural and civil engineering firm recognized for its expertise in designing resilient and sustainable projects. KPFF is dedicated to tackling complex challenges in urban development and structural systems. This project reflects that focus; it augments existing analysis tools to enhance usability and efficiency. The goal is to develop a user-friendly visualization tool for the OpenSees (Open System for Earthquake Engineering Simulation) software. OpenSees, stemming from academic research, boasts exceptionally powerful analysis tools, but lacks a user interface. Our software enhances the interpretation of structural and geotechnical analysis results and leverages OpenSees' analysis power for commercial engineering applications.

During natural disasters, it is imperative that critical emergency infrastructure, including data centers and community hospitals, remain operational. It is important to consider how the building moves under earthquake load, where structural damage occurs and to what extent, whether and how people can safely exit. Our project unites state-of-the-art analysis capabilities with intuitive visualization and simulation, supporting efforts in ensuring Los Angeles' architecture and infrastructure behave to the extent of resilience and safety required during seismic events.

2.2 Existing Solutions and Innovation

Current structural analysis software typically requires trade-offs between usability and analytical power. Some tools prioritize intuitive user interfaces, while others offer more advanced analysis capabilities at the expense of ease of use. Below are common structural analysis tools and their limitations:

- RISA-3D® is a commercial software with a clean user interface but limited functionality.
- ETABS® is a free, well-rounded software that supports many structural analysis tasks.
- Perform3D® is another commercial tool with stronger analytical capabilities, but its interface is outdated.
- OpenSees, the primary software we interacted with, is the most analytically powerful. However, this software does not have a user interface beyond the command-line.

KPFF's Current Structural Analysis Workflow:

1. Develop models that define the geometric and structural composition of buildings.
2. Generate .tcl files to represent these models and input them into OpenSees for analysis.
3. Apply seismic ground motion loads to simulate real-world earthquake scenarios.
4. Receive OpenSees analysis output as densely structured text files, which engineers must interpret manually. Alternatively, engineers can use Perform3D® for visualization and simulation, but it remains inadequate for their needs.

The lack of a visualization interface in OpenSees makes it tedious and unintuitive for engineers to analyze results efficiently. Other tools, such as Perform3D®, ETABS®, and RISA-3D®, lack either the analytical power of OpenSees or essential GUI functionality like interactive 3D manipulation (e.g., pan, zoom, rotate).

Currently, there is no satisfactory way to fully visualize or interact with OpenSees' analysis output. Our software addresses this limitation by integrating OpenSees with a user-friendly graphical interface, powered by Godot. This enables both pre- and post-processing visualization, real-time simulation, and more efficient workflows. By reducing manual effort and improving usability, VISTA makes OpenSees more practical for commercial applications.

2.3 Project Goals

The general project goal is to provide KPFF with a platform and functional software to visualize structures and buildings under earthquake simulations. VISTA streamlines their engineering pipeline, enhancing productivity and eliminating reliance on multiple tools, which can be inefficient and error-prone. Since many features depend on each other and are interconnected, we prioritized urgent needs while remaining flexible with lower-priority requests. The highest priority for KPFF is an effective interface that can visualize the post-processing analysis output from OpenSees for seismic events.

In more detail, this includes functionality for color coding damage areas based on severity and damage and displaying building deformations during earthquake simulations. VISTA integrates the advantages of other structural engineering tools with OpenSees' advanced analysis capabilities, complementing them with a modern and intuitive user interface.

2.4 Assumptions

We assume that VISTA users will provide the necessary data files for GUI visualization and analysis. This is an intentional design choice, ensuring flexibility while keeping VISTA lightweight. Required input files include pre-processing data defining 3D model geometry, elements, and properties, as well as files specifying ground motions for analysis. Since KPFF engineers will generate these files earlier in the workflow pipeline–often using tools like ETABS®–our focus remains on visualization and integration rather than file generation. The software processes these inputs and communicates seamlessly with OpenSees to facilitate real-time structural analysis.

Given the variety of sources, we anticipate that input files may be inconsistently formatted. VISTA is designed to handle such edge cases efficiently, ensuring robustness in parsing and processing. Additionally, with the adoption of Godot, we leverage its high-performance rendering system to enhance real-time visualization, eliminating the need for external libraries. Our confidence in Godot's capabilities comes from its dedicated 3D engine, flexible scripting via GDScript, and built-in support for an interactive GUI. These features align well with the project's visualization and usability goals.

We assume that users will have hardware capable of supporting 3D rendering. While VISTA is designed to be lightweight, performance may vary on lower-end systems. To assist in this area, we implemented initial performance optimizations to improve accessibility, analysis and smooth rendering for users working on less powerful hardware.

3. System Architecture

3.1 System Architecture Diagram

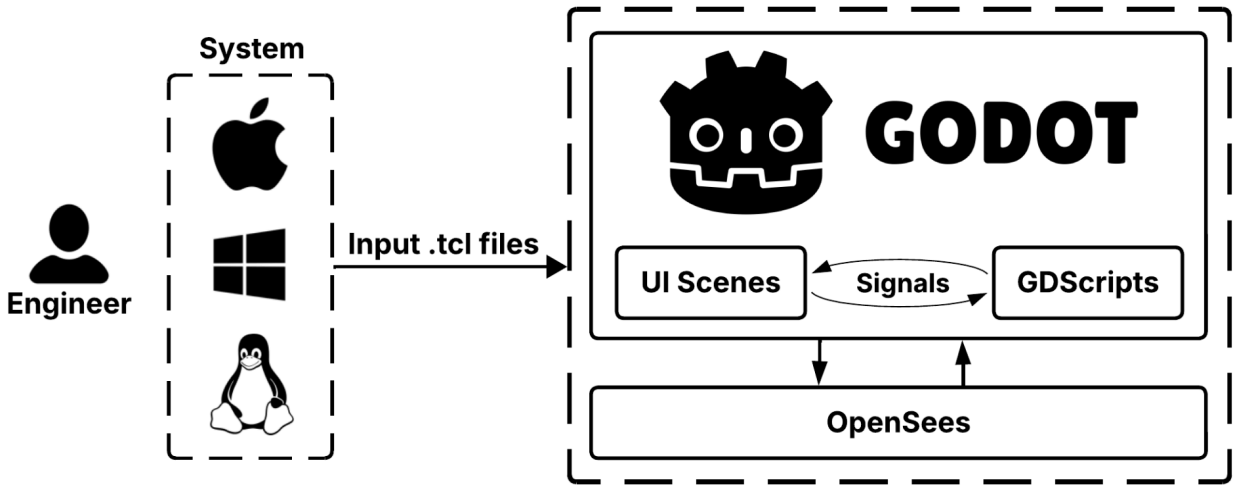


Figure 1. System architecture for VISTA software.

VISTA is powered by Godot, a feature-rich engine designed for interactive 2D and 3D applications, providing a unified development environment for both the frontend and backend through GDScript. With its comprehensive toolset and built-in cross-platform support, Godot eliminates the need for external GUI frameworks, ensuring seamless deployment across Windows, macOS, and Linux. Designed as a cross-platform visualization tool, VISTA handles and processes .tcl files in memory, with a potential for future enhancements such as a file management system. GDScript and Godot's scene system is used to parse, process, and visualize structural data in real time, enabling interactive, high-quality visualizations within the VISTA ecosystem. VISTA directly handles all data processing without external libraries, ensuring a streamlined workflow within Godot. Additionally, future development of VISTA will enable backend integration with OpenSees, allowing users to execute structural simulations directly within the software. The project was structured based on feature priority. This approach ensures a clear roadmap for future development and scalability, enabling the incremental addition of new functionalities over time.

3.2 GUI Design

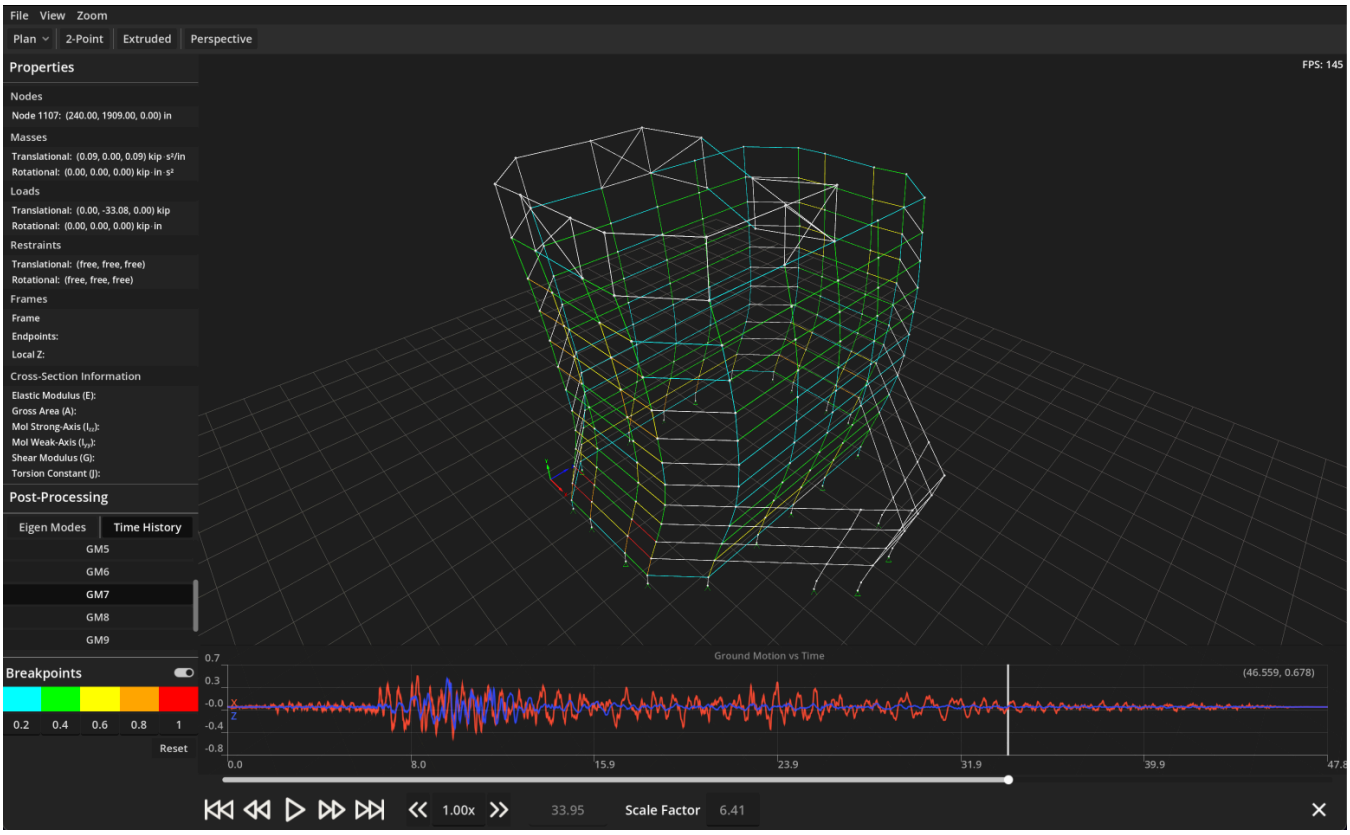


Figure 2. User interface workflow of VISTA, illustrating the 3D modeling view and various panels.

VISTA features a single primary page that serves as the main interface, with buttons leading to dropdown menus and pop-up windows for additional functionality. Since the core focus of the software is model visualization, all secondary windows are implemented as pop-ups rather than full-page views. This ensures that users can continue to interact with and observe the model while accessing various tools and settings, maintaining an uninterrupted workflow. The design was carefully planned using Godot's built-in GUI system, ensuring seamless integration with its 3D rendering capabilities. To achieve this, our initial design was modeled using Figma, drawing inspiration from established industry software such as Perform3D® and ETABS®. By adopting a familiar layout and interaction style, we provide users with a seamless transition, reducing the need for extensive onboarding or retraining. Dropdown menus provide quick access to key features, while pop-up windows allow users to modify parameters, run simulations and analyze results without losing sight of the model. Godot's real-time rendering improves visual feedback, ensuring VISTA remains intuitive and efficient for KPFF engineers.

5. Platforms and Technologies

5.1 Godot

Godot serves as both the frontend and backend of our software, handling the user interface, data processing and visualization. We leveraged its built-in GUI system and 3D rendering capabilities to create an interactive structural analysis tool. GDScripting manages data flow, user interactions and visualization logic, while GUI scenes communicate via signals to seamlessly integrate interface elements with OpenSees and the user. This unified approach eliminates the need for external backend processing, libraries or frameworks, and ensures real-time updates and a streamlined workflow.

5.2 ETABS®, RISA-3D®, and Perform3D®

We took inspiration from several industry-standard tools for structural analysis and building design. They serve as our primary references for developing VISTA. We take inspiration from their strengths while avoiding their limitations.

5.3 OpenSees

An open-source finite element analysis software used for simulating structural behavior under seismic loading. It provides powerful ground motion analysis capabilities but lacks a graphical user interface, requiring users to interact through a command-line environment.

5.4 GitHub

A private KPFF repository is being used for version control and collaborative development.

5.5 Jira

Our project management tool of choice to streamline all stages of development.

6. Project Implementation

6.1 Tasks and Milestones Completed

We first focused on establishing strong technical foundations. Our team completed KPFF's professional onboarding, including training on Microsoft Teams and internal collaboration tools. We quickly learned to navigate Jira for efficient task management and project tracking. A major technical milestone was gaining proficiency in both Godot and GDScript, which were entirely new platforms for us. Additionally, we analyzed the structure of OpenSees .tcl files and developed a custom interface to streamline interaction with simulation script data.

In terms of 3D visualization, we implemented foundational features such as gridlines to serve as a visual horizon and a 3D gizmo to mark the origin, both aiding engineers in model orientation. We built navigation functionality, allowing users to pan, rotate, and zoom using mouse and keyboard inputs. We also enabled hover and click detection for the sidebar property display, and rendered structural elements such as nodes and beams in iterative stages. Structural properties and restraints were assigned to elements with care to ensure accuracy and consistency.

For user interaction, we designed the overall layout by receiving weekly feedback from KPFF engineers. We introduced essential toggles and view options to tailor the engineer's experience. Various view modes were developed in iterative stages, as well as the ability to hide beams and columns. We added dropdowns for file uploading, feature viewing and zoom scaling. The zoom feature enables users to adjust viewing parameters and perspectives to personalize their workspace and optimize usability across systems.

On the pre- and post-processing side, we enabled script uploading for model creation, along with support for multiple viewing modes including plan, 2-point, wireframe, extruded, perspective, and isometric views. We visualized OpenSees output, which produced structural deformations, stress distributions, and failure zones after simulation. Demand-capacity ratio stress indicators were added using customizable color-coded markers, and seismic wave impacts were represented through a dynamic, togglable seismogram. Interactive playback controls were implemented to allow users to manage simulation progress, speed, and scaling.

6.2 Challenges and Roadblocks

Initially, we explored using Plotly, a Python graphing library, to visualize our 3D models. While Plotly allowed for rapid prototyping of visualization features, we quickly realized that its 3D rendering capabilities were limited. Our project required real-time rendering, interactive controls, and advanced visualization features, leading us to transition to Godot, a popular open source gaming engine. Godot provides a more robust rendering engine tailored for complex graphical applications, but adopting it required learning its interface, workflows, and native scripting language, GDScript. GDScript introduced a different syntax and structure, which required a learning curve before we could fully leverage its capabilities for structural analysis.

We also encountered challenges with parsing and processing .tcl script files provided by KPFF engineers, which contain structural models and load conditions for OpenSees simulations. We faced difficulties in understanding technical terminology and deciphering OpenSees-specific syntax embedded in these scripts. Ensuring accurate data extraction and seamless integration with Godot required careful script handling and team communication. Since KPFF envisions this as a multi-year project, we recognized the importance of thorough documentation. Our goal is to streamline the onboarding process for future developers, as well as users, to ensure the project's long-term sustainability and ease-of-use.

While our original roadmap included integrating OpenSees backend execution directly into the tool, KPFF ultimately chose to deprioritize backend integration in favor of focusing on user experience and 3D visualization features. This decision allowed us to concentrate our efforts on refining the front-end tool and delivering a more polished, interactive experience for engineers reviewing structural models.

7. Limitations and Future Work

While our team made substantial progress—often exceeding our own expectations during winter quarter—there are areas for improvement and further development as the project continues into future phases. One current limitation is our exclusive use of Godot and GDScript for the entire tech stack. While Godot has served us well for 3D visualization and rapid GUI development, its scripting environment may become a constraint when implementing more advanced features, such as backend computation or file management. A future improvement would involve integrating backend functionality, such as OpenSees simulation execution, with script export and build tools. This would require combining our front-end Godot system with a more robust processing backend to handle complex structural operations.

From a development standpoint, one challenge we encountered early on was the lack of structured sprint planning. During winter quarter, tasks were primarily self-assigned and implemented based on immediate needs rather than long-term roadmap alignment. Although we used Jira, it was often an afterthought rather than a planning tool. In response, we realigned our workflow for spring quarter, collectively agreeing to map out our remaining tasks and quarter-long milestones ahead of time. With a clearer understanding of the project's scope and future requirements, our sprint cycles significantly improved.

Looking ahead, KPFF has expressed interest in significantly expanding VISTA's capabilities. Planned features include group and elevation view modes, dynamic node and beam tag display, directional load arrows, and node/beam highlighting to enhance model clarity. Support for nonlinear elements—such as plastic hinges, damper elements, custom materials, and nonlinear shear walls—is also planned to allow more advanced structural modeling. To enhance analysis workflows, static pushover tools with automated capacity curve generation will be incorporated, along with FEMA/ASCE performance checks for drift limits and damage states. A 3D cityscape overlay is also envisioned to provide better project context for client presentations.

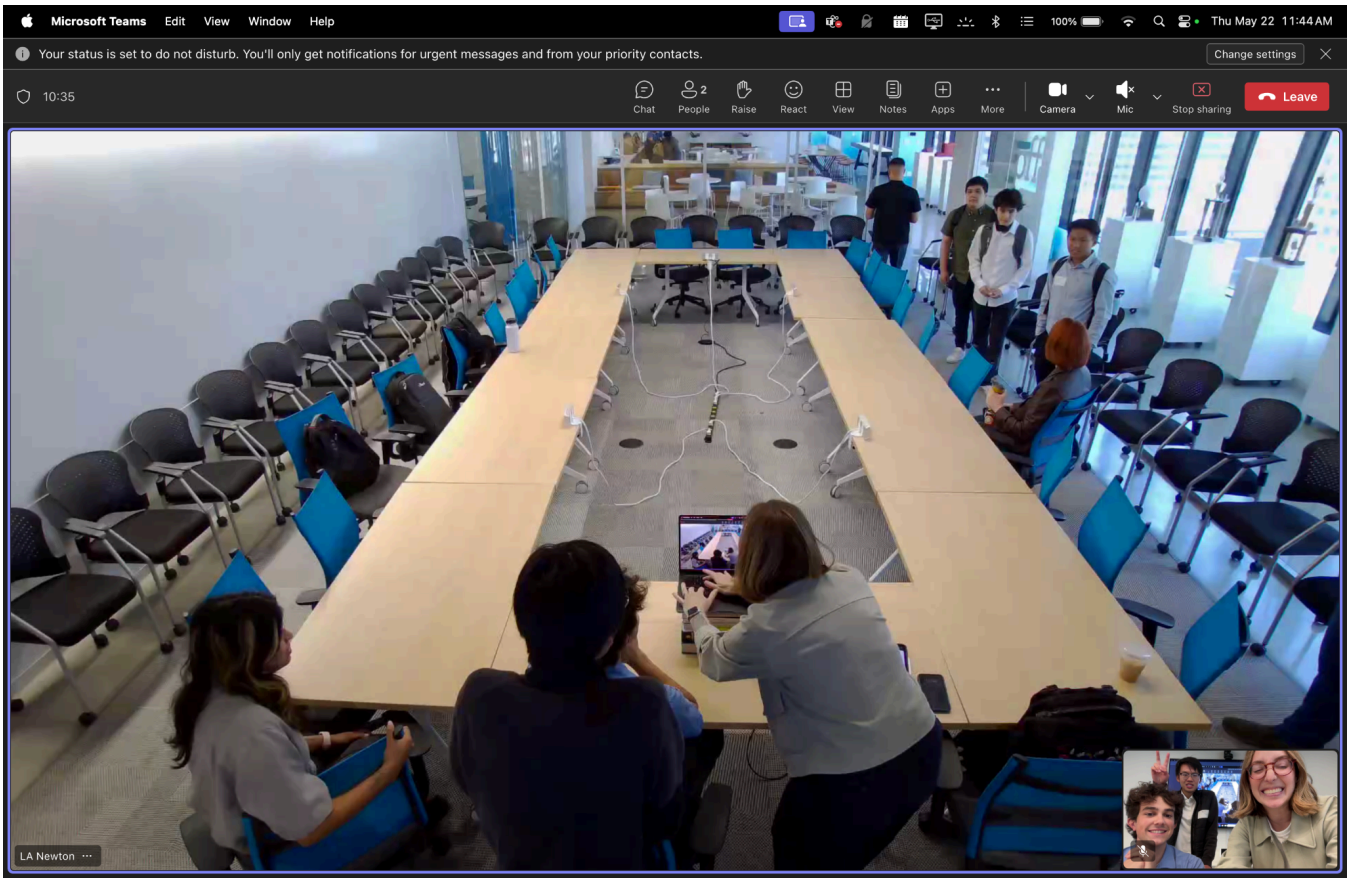
These future enhancements will not only expand the tool's technical capabilities but also improve usability, scalability, and integration across KPFF teams. By addressing current limitations and continuing to iterate with structured development practices, the tool is well-positioned for long-term success and broader adoption within the company. VISTA gives KPFF a competitive edge with a tool tailored to OpenSees, offering both research benefits and monetary advantage over peers. This software reinforces KPFF's innovation reputation, reflecting its commitment to engineering leadership and increasing visibility in research, recruiting, and expertise.

8. Deliverables

8.1 Demo Video

Our demo was completed in-person at the KPFF office in Downtown Los Angeles on May 22nd, 2025.

- KPFF Presentation Slides
- KPFF Presentation Demo



8.2 Source Code

Our Github repository is private, per KPFF's request. Code was shared throughout the process in one-on-one meetings with UCI staff.